Reproducibility Engineering
Summer Term 2026
Chair of Scalable Database Systems
Prof. Dr.-Ing. Stefanie Scherzinger, Stefan Klessinger, Christoph Köhnen

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

Exercise Sheet 1

Lab Sessions: April 23/24, 2026

1 Prerequisites

You are expected to have a working Docker setup when attending the first exercise session, either on your local machine or on our Docker server. If you have not yet done so, follow the instructions on the preliminary sheets available in StudIP.

The next exercise contains a short introduction to Docker. Students experienced with Docker can go on to Exercise 3.

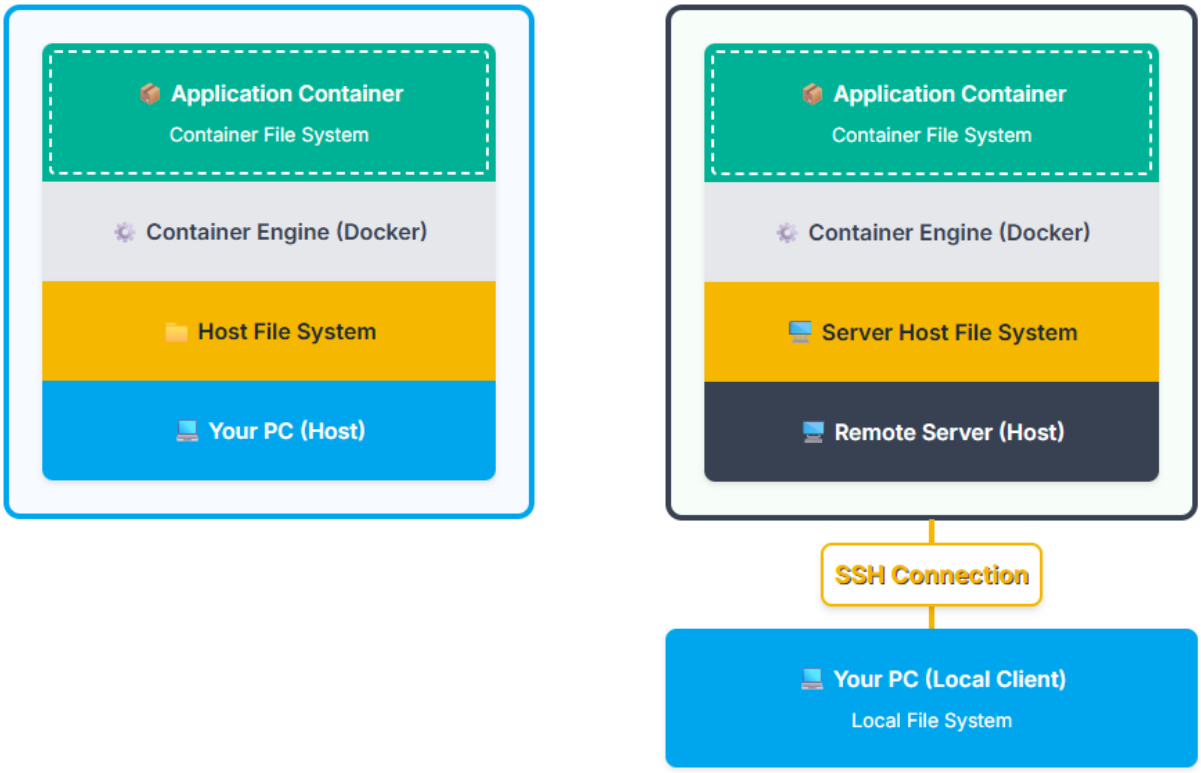
2 Introduction to Docker

Docker provides *lightweight virtualization*, enabling reproducible environments and a clear separation between an application and the underlying system. It is widely used to ensure that software runs consistently across different machines.

This virtualization is achieved through *containers*. A container bundles an application together with all its dependencies (e.g., libraries, tools, configuration files), ensuring that it behaves the same regardless of where it is executed.

(a) *Basic Concepts:*

- **Image:** A read-only template that defines the contents of a container (similar to a blueprint).
- **Container:** A running instance of an image. Containers are isolated from each other and from the host system.
- **Dockerfile:** A script containing the instructions to build an image.



(a) Local host machine (b) Docker server

Figure 1: Docker architecture on a local host machine and on our Docker server

(b) *Docker Architecture:* Docker follows a layered architecture (see Figure 1). At the top, the *application container* runs the user application inside an isolated *container file system*. This container is managed by the *container engine (Docker)*, which acts as an interface between containers and the host system.

Below the Docker engine lies the *host file system*, which belongs to the underlying machine (the *host*). If Docker is executed locally, this host is your own computer (see Figure 1a). If a remote Docker server is used, the host is a remote machine, which can be accessed from your local computer via an *SSH connection* (see Figure 1b).

In both cases, containers share the host's operating system kernel, but remain isolated through their own file systems.

(c) *Working with Containers:*

- **Building images:** Using `docker build` and a Dockerfile.
- **Running containers:** Using `docker run`.
- **Copying files:** Using `docker cp` to transfer files between host and container.
- **Bind mounts:** Sharing directories between host and container using `-v`.

3 Setting Up a Docker Container

In this exercise, we set up a Docker container which will be used in the following exercises on this sheet.

3.1 Notation

We will use the following notation for command-line tasks. Do *not* copy the prefixes to the command line; they are just an indicator of *where* to execute the command.

- `user@host$`: Execute the command on the *host* machine.
- `user@container$`: Execute the command inside the *Docker container*.

The *host* machine is the machine on which the Docker container runs. The two relevant scenarios for this course are illustrated in Figure 1. If you run Docker on your own machine, your computer is the *host* machine (see Figure 1a). If you are using the Docker server, the server is the *host* machine (see Figure 1b).

3.2 Building an Image

Clone the RepEng FIMGit Repository:

```
user@host$ git clone https://git.fim.uni-passau.de/koehnen/RepEng
```

Change the directory to the LabSession1 sub-directory:

```
user@host$ cd RepEng/LabSession1
```

Open the Dockerfile with a text-editor, e.g.:

```
user@host$ nano Dockerfile
```

Inspect the content of the file. In the following steps, we build a Docker image using this Dockerfile as a construction plan.

Build the Docker container with the provided Dockerfile with a tag `lab1` using the parameter `-t`:

```
user@host$ docker build -t lab1 .
```

Check whether the image is built, using the following command:

```
user@host$ docker image ls
```

For example, if the image has the ID `7bfe9025db3f` and was created 13 seconds ago, the output is as follows:

```
REPOSITORY   TAG      IMAGE ID       CREATED          SIZE
lab1         latest   7bfe9025db3f   13 seconds ago   423MB
```

3.3 Running a Container from an Image

After building the image, we set up a Docker container from that image.

(a) Run the Docker container from the image we built before (with tag `lab1`) in interactive mode. For convenience, name the container `lab1-cont` using the optional parameter `--name`:

```
user@host$ docker run -it --name lab1-cont lab1
```

You should see a command prompt similar to the following one:

```
repro@48e05f93af51:~\$
```

What do its elements mean?

(b) You are now inside the Docker container. Check the contents of the working directory:

```
user@container$ ls -l
```

Which files are located in the working directory?

Exit the container using the command `exit`.

4 Working with Docker Containers: Image Comparison with Python

Inside the repository in the folder `LabSession1/res/`, you find two images. One of these images has a secret message embedded (`*_secret.jpg`).

The goal of this task is to explore different approaches to determine whether the two images are identical or not. To answer this question, we will apply different comparison methods.

This exercise not only introduces different techniques for comparing files, but also serves as a first example of working with Docker containers.

Change the directory to the LabSession1 sub-directory.

(a) We start with a simple visual inspection and basic file properties. Open the images and compare them visually. Also compare the timestamps and sizes of the files. What do you observe?

(b) Next we perform a bitwise comparison. Compute the SHA-256 checksums as follows:

```
user@host$ sha256sum res/fox.jpg
user@host$ sha256sum res/fox_secret.jpg
```

What do you observe?

(c) Finally, we perform a statistical analysis of the image content. We write a Python script inside the Docker container that measures how similar the two images are by computing the *Pearson correlation*[1] of their pixel values.

First, prepare the Docker container. Check whether the Docker container we created in Exercise 3 is running using the command `docker ps`. If `lab1-cont` does not appear in the list, start the container:

```
user@host$ docker start lab1-cont
```

Access the container by launching an interactive shell:

```
user@host$ docker exec -it lab1-cont bash
```

Next, install packages in the container that allow us to read and manipulate image data in Python and calculate the correlation:

```
user@container$ pip3 install --user numpy pillow
```

Finally, write a Python script `correlation.py` that computes the Pearson correlation between the two images. Execute it as follows:

```
user@container$ python3 correlation.py
```

What is the computed correlation value and what does this mean?

Hints:

- Use the installed command-line text editor *nano* to create and edit the Python script directly inside the container.

  ```
  user@container$ nano correlation.py
  ```

  Optional: You can install and use other command-line text editors like *micro* or *vim* with the command "`sudo apt install <name>`" replacing `<name>` by, e.g., `micro` or `vim`. The password is "repro".

- Follow these steps to write the script:
  - Load both images using the function `Image.open`.
  - To simplify the comparison, convert the images into grayscale arrays using the function `Image.convert`.
  - Turn the images into `NumPy` arrays using the function `numpy.array`.
  - Flatten the 2-dimensional image arrays into 1-dimensional pixel vectors using the function `numpy.flatten`.
  - Compute the Pearson correlation between the two pixel vectors using the function `numpy.corrcoef`.

[1] Details about the Pearson correlation: https://www.sciencedirect.com/topics/computer-science/pearson-correlation

(d) As the host system and the Docker container have different file systems, a way to transfer files between these two systems might be convenient. This can be achieved through the command `docker cp`. We copy the Python script to the host, edit it, and copy the new version into the container as follows.

Copy the file to your host system (into a directory of your choice, e.g., `~`):

```
user@host$ docker cp lab1-cont:/home/repro/correlation.py ~
```

On your host system, add the following line to the file:

```
print("Edited on host.")
```

Copy the edited file to the container:

```
user@host$ docker cp ~/correlation.py lab1-cont:/home/repro/correlation.py
```

Access the container:

```
user@host$ docker exec -it lab1-cont bash
```

Execute the script:

```
user@container$ python3 correlation.py
```

(e) A more convenient way to synchronize files on your host machine with the container is using bind mounts which map a host directory inside the container.

As mounts must be created during container setup, spin-up a new container:

```
user@host$ docker run -it --name lab1-cont-mount \
           -v /home/user/lab1-mount:/home/repro/mount lab1
```

Replace `/home/user/lab1-mount` with an absolute path on the host file system. In the following, we assume that the host directory `/home/user/lab1-mount` is mapped to the container. Now, you can upload files to the folder `/home/repro/mount` in the Docker container by moving them into `/home/user/lab1-mount` (on the host system).

On the host system, create a file `test.sh` in the directory `/home/user/lab1-mount` with the following content:

```
echo "Test passed!"
```

Start (if necessary) and enter the container as described in (c). Then, grant execution permissions to the file (password is `repro`):

```
user@container$ sudo chmod +x mount/test.sh
```

Execute the file:

```
user@container$ ./mount/test.sh
```

5 Working with Docker Containers (Multiple Choice, Antwort-Wahl-Verfahren)

In this exercise, each question has exactly one correct answer. Mark the correct answer.

Consider the following Dockerfile:

```
FROM ubuntu:24.04

RUN apt update && apt install -y bash coreutils lsb-release
COPY experiment.sh /usr/local/bin/experiment.sh
RUN chmod +x /usr/local/bin/experiment.sh
```

File `experiment.sh` contains the following script:

```
#!/bin/bash

echo "Starting experiment..."

a=7
b=5
sum=$((a + b))

echo "Input values: a=$a, b=$b"
echo "Sum: $sum"
# lsb_release -ds shows the name of the Linux distribution
echo "Linux distribution: $(lsb_release -ds)"

echo "Experiment finished."
```

Alice and Bob each execute the following commands on their own machines. Alice uses a laptop with Debian Linux 13 and an Intel Core i7 CPU. Bob uses a desktop PC with Ubuntu Linux 25 and an AMD Ryzen 9 CPU.

```
user@host$ docker build -t experiment-app .
user@host$ docker run -itd --name exp-cont experiment-app
user@host$ docker exec exp-cont /usr/local/bin/experiment.sh
```

(a) Which statement is correct?

- ○ They may get different results because they use different Linux distributions.
- ○ They will get the same output because the script is deterministic and the container provides the same environment.
- ○ They will get different output because the image is rebuilt on each machine.
- ○ None of these options

(b) Which command shows that the file `experiment.sh` exists inside the container?

- ○ `user@host$ ls /usr/local/bin/experiment.sh`
- ○ `user@host$ ls exp-cont:/usr/local/bin/experiment.sh`
- ○ `user@host$ docker exec exp-cont ls /usr/local/bin/experiment.sh`
- ○ `user@container$ ls /home/user/experiment.sh`
- ○ None of these options

(c) Alice executes the following command:

```
user@host$ docker cp exp-cont:/usr/local/bin/experiment.sh \
                     experiment_copy.sh
```

Which command shows the file `experiment_copy.sh`?

- ○ `user@host$ ls experiment_copy.sh`
- ○ `user@host$ docker exec exp-cont ls /usr/local/bin/experiment_copy.sh`
- ○ `user@container$ ls /usr/local/bin/experiment_copy.sh`
- ○ `user@host$ ls exp-cont:/usr/local/bin/experiment_copy.sh`
- ○ None of these options

Production of this lecture material was financially supported by Lehrinnovationspool 2.0/2021-2022 at the University of Passau. The content was developed in collaboration with Wolfgang Mauerer, OTH Regensburg, Germany.
Parts of the content were modified by Stefanie Scherzinger.